目录

背景与动机

为什么需要Morris算法？解决传统遍历的痛点

核心思想

线索二叉树的巧妙运用，利用空闲指针

算法详解

以中序遍历为例，分步拆解执行逻辑

实例演示

通过具体例子理解算法执行过程与状态变化

变种拓展

前序与后序遍历的Morris实现方式

总结与思考

算法优缺点分析与实际应用场景探讨

背景与动机：我们为什么需要Morris?

常规遍历方法

- **递归：**简单直观，但有栈溢出风险，空间复杂度O(h)。
- **迭代：**手动模拟栈，规避溢出，空间复杂度仍为O(h)。

痛点与需求

- **共同痛点：**深度极大时内存开销大。
- **核心需求：**在嵌入式等受限环境中，迫切需要**O(1)**空间复杂度。

Morris算法

- **应运而生：**解决O(h)空间瓶颈。
- **核心优势：**通过线索化，实现**O(n)**时间复杂度与**O(1)**空间复杂度。

总结：Morris算法并非为了取代传统方法，而是为了解决传统方法在处理极深二叉树时的内存瓶颈问题，是一种极具巧思的优化方案。

核心思想：线索二叉树的巧妙运用

空间的“浪费”

普通二叉树中存在大量空指针（NULL），占用了存储空间却未被有效利用。

Morris的智慧

利用空闲的空指针建立临时“线索”（Thread），构建无需栈的遍历路径。

线索的价值

将前驱节点的Right指针指向当前节点，就像“**记住了回家的路**”，从而省去了栈的开销。

Morris 遍历关键步骤

1. 建立线索 (Threading)

首次访问节点时，找到其中序前驱节点，将其Right指针指向当前节点。

2. 拆除线索 (Unthreading)

第二次访问节点（通过线索返回时），将前驱节点的Right指针恢复为NULL，还原树结构。

算法详解：以中序遍历为例

1. 初始化

当前节点 cur 指向根节点，准备开始遍历。

2. 无左孩子 (直接访问)

访问 cur，然后 cur 向右移动 (cur = cur->right)。

3. 有左孩子 (找前驱节点)

找到左子树最右节点 prev：

- prev->right == NULL：建立线索，cur 左移
- prev->right == cur：拆除线索，访问 cur，右移

4. 循环终止：直到 cur == NULL

Morris 核心思想：线索化

利用叶子节点的空指针，指向其后继节点，构建“线索”。

避免了递归或栈带来的额外空间开销，实现**O(1)**空间复杂度的遍历。

算法伪码示意

```
while (cur is not null):
if (cur.left is null):
visit(cur); cur = cur.right;
else:
find predecessor...
if (pred.right is null): ...
else: ... // 恢复线索
```

实例演示：一步一步走一遍

图示：标准二叉树结构 (根节点为1)

结构定义

我们以一个简单的二叉树为例：根节点为 1，左子树包含 2、4、5，右子树包含 3。

预期中序遍历结果

4 → 2 → 5 → 1 → 3

演示目标：接下来，我将带着大家一步一步地走一遍 Morris 算法的执行过程，重点观察它是如何不使用栈（O(1) 空间复杂度）就完成上述中序遍历的。

实例演示：步骤 1 - 4

二叉树线索化状态变化示意

步骤 1：cur = 1，有左孩子。找到左子树(2)最右节点(5)，其right为NULL。建立线索：5->right = 1。cur = 2。

步骤 2：cur = 2，有左孩子。找到左子树(4)最右节点(4)，其right为NULL。建立线索：4->right = 2。cur = 4。

步骤 3：cur = 4，无左孩子。访问 4。cur = 4->right = 2。

步骤 4：cur = 2，有左孩子。发现线索已建立，拆除线索(4->right=NULL)。访问 2。cur = 2->right = 5。

实例演示：步骤 5 - 7

步骤 5：访问节点 5

cur = 5，无左孩子，直接访问。随后 cur = 5->right = 1。

步骤 6：拆除线索，访问节点 1

cur = 1，发现左子树线索已建。拆除线索 5->right = NULL，访问 1。cur = 1->right = 3。

步骤 7：访问节点 3，遍历结束

cur = 3，无左孩子，访问 3。cur = 3->right = NULL，遍历结束。

最终访问顺序：4 → 2 → 5 → 1 → 3

图示：步骤 5-7 的二叉树状态变化与线索

变种拓展：前序与后序遍历

Morris算法的核心思想可以轻松扩展到前序和后序遍历，主要区别在于**访问节点的时机**。

前序 Morris 遍历

访问时机：建立线索时（即 prev->right == NULL）访问当前节点 cur。
口诀调整：有左孩子且 prev->right == NULL 时，先访问 cur，再建立线索。

后序 Morris 遍历

访问时机：第二次访问节点（拆除线索时），逆序输出其左子树的右边界。
实现思路：可通过临时哨兵节点，将整棵树作为其左子树，然后统一处理。

核心总结：掌握中序遍历后，变种算法的关键在于灵活调整访问节点的时机，无论是建立线索时的“顺手牵羊”，还是拆除线索时的“回溯处理”。

总结与思考：优缺点与适用场景

核心优势

- **O(1) 空间复杂度**
  不使用栈或递归，仅利用树空指针
- **时间效率高**
  复杂度O(n)，节点最多访问三次
- **无栈溢出风险**
  避免深度过大导致的递归栈溢出

主要不足

- **代码实现复杂**
  逻辑晦涩，编写与理解难度大
- **修改树结构**
  临时修改指针，并发环境不安全
- **可读性较差**
  算法不直观，维护成本相对较高

适用场景

- **资源受限环境**
  如嵌入式系统，对内存极度敏感
- **极深二叉树**
  无法承受递归栈开销的处理场景
- **算法面试考点**
  考察对二叉树与指针操作的理解

Morris算法总结：这是一种“以时间换空间”的极致优化算法。虽然牺牲了一定的代码可读性和直观性，但在内存资源极度紧张的场景下，它是实现线性时间遍历二叉树的最佳选择。